

2026-05-06-p1-5-foundation-cleanup

Phase 1.5 — Foundation Cleanup Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (- []) syntax for tracking.

Programme position: Phase 1.5 is the bridge between Phase 1 (CLI-Agent Fusion features F01–F20, complete at commit 046d802) and Phase 2 (other CLI agents). It is NOT optional. The audit performed by the user revealed structural debt that, if left unresolved, would multiply across every Phase 2/3 submodule we touch (duplicate submodules at multiple paths, ~60 cli_agents/ entries trapped under HelixAgent including a circular self-reference, three competing Documentation/ trees, eight .env files with API-key duplication, ad-hoc directory casing). Phase 1.5 does the surgery once so every later phase compounds clean.

Goal: Land a verifiably-clean, deduplicated, snake-cased, anti-bluff-anchored repository topology with a single canonical secret-loading path (api_keys.sh → .env fallback), one docs/ tree per repo, root-level cli_agents/ as the single home for all CLI-agent submodules, and a Phase 1.5 Challenge harness that fails any future regression.

Scope (12 work packages, ~110 tasks total):

WP	Title	Tasks	Deepest risk
WP1	Inventory + foundation safety	5	accidental destructive act before snapshot
WP2	Submodule restructuring	~67	brick the meta-repo’s submodule tree mid-move
WP3	Submodule deduplication	5	break consumers expecting old paths
WP4	API key loader (bash + Go)	6	inconsistent loader behaviour across repos
WP5	.env API key dedup (USER GATE)	4	destructive secret removal without verified loader
WP6	Docs consolidation	3	broken internal links across docs
WP7	Snake_case directory normalization	~10–20	case-collision on case-insensitive FS
WP8	Anti-bluff Constitution propagation	3	repos drift from canonical anchors

WP	Title	Tasks	Deepest risk
WP9	Reference updates (catch-up sweep)	2	stale references to old paths
WP10	Rebuild + validation	8	undetected build/test regression
WP11	Phase 1.5 Challenge harness	4	bluffy harness (passes without evidence)
WP12	Commit + push everything	5	push-order dependency cascade

Success criteria (each independently verifiable):

1. `git submodule status --recursive` reports each canonical URL at exactly one path (Phase A of harness).
2. `scripts/load_api_keys.sh` and `internal/secrets/loader.go` both source `~/api_keys.sh` when present, else fall back to local `.env`, else warn no-op (Phase B).
3. No `Documentation/` directory remains anywhere in tree; each repo has its content under `docs/` (Phase C).
4. Every directory under tracked content (allowlist applied) matches `^[a-z][a-z0-9_]*$` or is a known repo-name root (Phase D).
5. `CONST-035 + Article XI §11.9` anchor present in every `Helix*` submodule's `CONSTITUTION.md` (Phase E).
6. `cd HelixCode && make verify-compile && make test` passes; `internal/tools/git mock drift fixed` (T10.08).
7. All four meta-repo remotes (`origin/github/gitlab/upstream`) advanced to the same SHA; each `Helix*` submodule pushed to all its configured remotes.
8. Anti-bluff smoke command at the bottom of this plan returns `clean` after every WP closes.

Tech Stack: `bash` (POSIX-portable), `git submodule`, `git mv`, `git rm --cached`, GNU `sed/grep`, `ripgrep (rg)`, Go 1.26 (for the loader and harness), `testify`, GNU Make. **Zero new external dependencies.**

Working directory: `/run/media/milosvasic/DATA4TB/Projects/HelixCode` (meta-repo root). Inner Go module commands run from `helix_code/`.

Anti-bluff smoke (run at the END of every task):

```
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
  helix_code/internal helix_code/cmd 2>/dev/null \
  && echo "BLUFF FOUND" || echo "clean"
```

Must always print `clean`.

Cross-cutting constraints

- **CONST-035 / Article XI §11.9 (Anti-Bluff Forensic Anchor):** every PASS in this plan must carry positive runtime evidence. Metadata-only / configuration-only / absence-of-

error / grep-based PASS without runtime evidence is a critical defect. The Phase 1.5 Challenge harness (WP11) is the enforcement gate.

- **CONST-042 (No-Secret-Leak):** WP4 + WP5 are the practical implementation. `~/api_keys.sh` (mode 0600, listed in user's home, never committed) is the canonical key store; local `.env` files (mode 0600, listed in `.gitignore`) hold non-overlapping config only after WP5.
 - **CONST-043 (No-Force-Push):** every `git push` in this plan is non-force. WP12 push order is deepest-first to avoid forcing parents to catch up. Any `--force / --force-with-lease` requires explicit per-operation user authorisation (none granted here).
 - **CONST-033 (Host Power Management — Hard Ban):** no `systemctl suspend`, `shutdown`, `reboot`, `loginctl suspend`, or any host-power command appears anywhere in this plan or any artefact it produces.
 - **No CI/CD pipelines (Rule 1):** WP11 harness is a Go program invoked manually or via Makefile; no GitHub Actions / GitLab CI artefact is created.
-

Risks (read these BEFORE starting any WP)

1. **WP2 — submodule-tree fragility.** ~60 sequential `git submodule deinit + git rm --cached + git submodule add` operations. One missed step can leave dangling gitlinks that break `git submodule update --init --recursive` for fresh clones. **Mitigation:** WP1.T01.02 captures a full snapshot to `docs/improvements/p1-5-snapshot-pre.md`; one submodule per commit; verify each commit with `git submodule status` before moving to the next.
 2. **WP3 — consumer breakage.** Removing the HelixAgent-internal copy of e.g. `LLMsVerifier` may break code that does `import "../..helix_agent/LLMsVerifier/..."` or `cd helix_agent/LLMsVerifier && make`. **Mitigation:** comprehensive `rg` for the old path BEFORE removal; build-test the affected repo after each dedup; rollback recipe documented per-WP.
 3. **WP5 — destructive secret removal.** Removing keys from `.env` without a verified loader leaves the user unable to run anything. **Mitigation:** explicit USER GATE at T05.01 (the agent stops and asks); per-key dry-run report listing exactly which keys-to-remove and which file each is in; only proceed on user OK.
 4. **WP7 — case-rename on case-insensitive filesystems.** macOS default APFS is case-insensitive; `git mv Foo foo` looks like a no-op. **Mitigation:** all renames done on Linux (the working host is Linux 6.12 per env); document that contributors on macOS must use `git config core.ignorecase false` and clone fresh after Phase 1.5 lands.
 5. **WP12 — push-order cascade.** Pushing a parent before its child means the parent points at a SHA the remote doesn't have; the push succeeds but other clones break. **Mitigation:** deepest-first push order (`cli_agents/` → `helix_agent/HelixQA` → `HelixAgent` → `helix_qa root` → `meta-repo`); per-remote `git ls-remote` verification at T12.05.
-

Estimated effort

3–4 focused sessions if executed cleanly. WP2 alone is ~60 mechanical commits; budget one full session for WP2 + WP3. Budget one session for WP4–WP6 (loader + `.env` dedup gated on user, docs merge). Budget one session for WP7 + WP8 + WP9 (normalization + anchor cascade + reference catch-up). Budget one session for WP10 + WP11 + WP12 (rebuild/validate + harness + push).

Open issues acknowledged as TBD-during-execution

Item	Why deferred	Resolution point
mcp_servers canonical location	unclear whether root mcp_servers/ exists yet, or whether to promote from one of the two HelixAgent locations	T03.05 picks at execution time; record in evidence
Example_Projects/ submodule count	not enumerated in inventory; deletion may be ~40 separate git rm --cached operations	T02.64 enumerates first, then deletes
WP2 collision count	overlap between Example_Projects/ contents and root cli_agents/ unknown until snapshot	T02.65 reconciles per-collision
WP7 directory count	non-conforming dirs not pre-counted across all submodules	T07.01 inventories first, T07.02..T07.NN per finding
Pre-existing internal/tools/git mock drift	known broken since F09; specific drift unknown	T10.08 reads the failing build, fixes mock to match interface
Per-submodule remote count	varies (Challenges has 1, meta has 4, others between)	T12.02 reads each .git/ config at push time

Cross-cutting conventions for every task

- **Branch:** stay on main. Phase 1.5 is foundation; per-task feature branches would multiply the ~110-task overhead.

- **Commit format:**

<type>(P1.5-<WP>-T<NN>): <subject>

<short description>

Phase: P1.5

WP: <WP> — <title>

Task: P1.5-<WP>-T<NN>

Evidence: <pasted command output OR pointer to docs/improvements/07_ph

Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>

- **Push:** per WP12 — deepest-first, all configured remotes per repo, non-force only.
- **PROGRESS.md:** every task closes by advancing docs/improvements/PROGRESS.md.
- **Anti-bluff smoke:** run after every task; commit blocks if it prints BLUFF FOUND.

Work Package 1 — Inventory + foundation safety (5 tasks)

Goal: Make Phase 1.5 reversible. Capture the pre-state in enough detail that any later mistake can be diagnosed and rolled back without blind reconstruction.

P1.5-WP1-T01.01 — Fetch + pull every submodule recursively

```
git submodule update --init --recursive --remote --jobs 8 2>&1 |
tee /tmp/p1-5-fetch.log
git submodule foreach --recursive
'git fetch --all --prune --tags 2>&1' | tee -a /tmp/p1-5-
fetch.log
git submodule status --recursive | tee /tmp/p1-5-initial-shas.txt
```

Record initial gitlink SHAs into docs/improvements/p1-5-snapshot-pre.md \$
“Initial SHAs”.

Commit: chore(P1.5-WP1-T01.01): record initial submodule SHAs pre-cleanup.

P1.5-WP1-T01.02 — Snapshot full submodule tree state

Write docs/improvements/p1-5-snapshot-pre.md with: - Full .gitmodules content of meta-repo + HelixAgent + every other parent that has one. - git submodule status --recursive output. - git status per submodule (a one-pass foreach). - The 5 duplicate sets and their decided canonical paths (from inventory). - The full list of cli_agents/ entries in HelixAgent (~60). - Rollback recipe: how to restore from this snapshot if WP2 breaks.

Commit: docs(P1.5-WP1-T01.02): pre-cleanup snapshot for rollback reference.

P1.5-WP1-T01.03 — Verify all submodule remotes are reachable

```
git config -f .gitmodules --get-regexp '^submodule\..*\\.url$' \
| awk '{print $2}' | sort -u | while read url; do
printf "%-80s " "$url"
git ls-remote "$url" HEAD >/dev/null 2>&1 && echo OK || echo
UNREACHABLE
done | tee docs/improvements/p1-5-remote-reachability.md
```

Same for HelixAgent's .gitmodules. Any UNREACHABLE blocks WP2 for that submodule until resolved (out-of-scope fix recorded in PROGRESS.md parking lot).

Commit: docs(P1.5-WP1-T01.03): verify submodule remote reachability.

P1.5-WP1-T01.04 — Construct + commit the deduplication decision list

Write docs/improvements/p1-5-dedup-decisions.md:

Submodule	Canonical path	Removed paths	Consumers to update
LLMsVerifier	submodules/ llms_verifier	helix_agent/ LLMsVerifier	helix_agent/ Makefile, helix_agent/scripts/*, helix_agent/internal references
containers	containers/	3 nested copies (TBD enumerate)	TBD per consumer
Security	security/	2 nested copies (TBD enumerate)	TBD per consumer
helix_qa	helix_qa/	helix_agent/ HelixQA	helix_agent/ Makefile, HelixAgent test wiring
mcp_servers	TBD at T03.05	TBD	TBD

Commit: docs(P1.5-WP1-T01.04): submodule deduplication decisions.

P1.5-WP1-T01.05 — Bootstrap evidence + advance PROGRESS

Create docs/improvements/07_phase_1_5_evidence.md with section headers for WP1..WP12. Update docs/improvements/PROGRESS.md: active phase → P1.5; active WP → WP1; advance task list.

Commit: docs(P1.5-WP1-T01.05): bootstrap Phase 1.5 evidence + advance PROGRESS.

Work Package 2 — Submodule restructuring (~67 tasks)

Goal: Move every cli_agents/ submodule from helix_agent/cli_agents/ to root-level cli_agents/. Eliminate the circular helix_agent/cli_agents/HelixCode entry. Move cli_agents_configs/ (plain content) and Example_Resources/ (rename to cli_agents_resources/). Delete Example_Projects/ after collision reconciliation.

P1.5-WP2-T02.01 — Delete circular cli_agents/HelixCode entry

The meta-repo cannot be its own grandchild. From helix_agent/:

```
cd HelixAgent
git submodule deinit -f cli_agents/HelixCode 2>/dev/null || true
git rm --cached cli_agents/HelixCode 2>/dev/null || true
git config -f .gitmodules --remove-section submodule.cli_agents/
    HelixCode 2>/dev/null || true
rm -rf .git/modules/cli_agents/HelixCode
git add .gitmodules
git commit -m "chore(P1.5-WP2-T02.01): remove circular
    cli_agents/HelixCode submodule entry"
cd ..
git add HelixAgent
git commit -m "chore(P1.5-WP2-T02.01): bump HelixAgent gitlink
    after circular removal"
```

Verify: `git submodule status --recursive | grep -c "cli_agents/HelixCode"` returns 0.

P1.5-WP2-T02.02 .. P1.5-WP2-T02.61 — Move each cli_agents/<NAME> to root cli_agents/

For each of the ~60 entries (enumerate from helix_agent/.gitmodules), one task per submodule. Per-task pattern:

```
NAME=<entry-name>
URL=$(git config -f helix_agent/.gitmodules --get
    "submodule.cli_agents/${NAME}.url")
SHA_BEFORE=$(cd "helix_agent/cli_agents/${NAME}" &&
    git rev-parse HEAD)

# 1. Inside HelixAgent — deinit + remove
(cd HelixAgent && \
    git submodule deinit -f "cli_agents/${NAME}" && \
    git rm --cached "cli_agents/${NAME}" && \
    git config -f .gitmodules --remove-section
        "submodule.cli_agents/${NAME}" && \
    rm -rf ".git/modules/cli_agents/${NAME}")

# 2. Commit inside HelixAgent
(cd HelixAgent && git add .gitmodules && \
    git commit -m "chore(P1.5-WP2-T02.NN): remove cli_agents/$
        {NAME} (moving to meta-repo root)")

# 3. Add at meta-repo root, pinning to the same SHA
```

```

git submodule add "$URL" "cli_agents/${NAME}"
(cd "cli_agents/${NAME}" && git checkout "$SHA_BEFORE")

# 4. Verify gitlink SHA matches pre-state
SHA_AFTER=$(cd "cli_agents/${NAME}" && git rev-parse HEAD)
[ "$SHA_BEFORE" = "$SHA_AFTER" ] || { echo "SHA DRIFT for
    $NAME"; exit 1; }

# 5. Bump HelixAgent gitlink + commit meta-repo
git add HelixAgent .gitmodules "cli_agents/${NAME}"
git commit -m "chore(P1.5-WP2-T02.NN): move cli_agents/$
    {NAME} to meta-repo root @ ${SHA_BEFORE:0:12}"

```

Commit one submodule per commit. Numbering: T02.02 .. T02.61 corresponds to alphabetical ordering of the 60 entries (record exact order in docs/improvements/p1-5-wp2-order.md at start of WP2).

P1.5-WP2-T02.62 — git mv helix_agent/ cli_agents_configs cli_agents_configs

```

git mv helix_agent/cli_agents_configs cli_agents_configs
git commit -m "chore(P1.5-WP2-T02.62): move cli_agents_configs
    (plain content) to meta-repo root"

```

P1.5-WP2-T02.63 — Rename Example_Resources/ → cli_agents_resources/

```

git mv Example_Resources cli_agents_resources
git commit -m "chore(P1.5-WP2-T02.63): rename Example_Resources
    -> cli_agents_resources"

```

P1.5-WP2-T02.64 — Delete Example_Projects/

Enumerate first:

```

git config -f .gitmodules --get-regexp
    '^submodule\.Example_Projects/.*\.path$' \
    | awk '{print $2}' > /tmp/p1-5-example-projects.txt
wc -l /tmp/p1-5-example-projects.txt

```

For each path, verify it is genuinely redundant with the new root cli_agents/<NAME> (compare URLs). If not redundant, parking-lot it and skip.

Per-path removal (one commit each, OR batched if all genuinely redundant):

```

while read p; do
    git submodule deinit -f "$p"
    git rm --cached "$p"

```



```

git config -f .gitmodules --remove-section "submodule.$p"
rm -rf ".git/modules/$p"
done < /tmp/p1-5-example-projects.txt
git rm -r Example_Projects
git add .gitmodules
git commit -m "chore(P1.5-WP2-T02.64): delete Example_Projects
(redundant with root cli_agents/)"

```

P1.5-WP2-T02.65 — Reconcile collisions

Detect: any submodule that ended up at TWO paths (root cli_agents/ AND something else).
Decide canonical, remove the other:

```

git submodule status --recursive | awk '{print $2}' \
| sort | uniq -c | awk '$1 > 1' | tee /tmp/p1-5-collisions.txt

```

Per collision: keep root cli_agents/; remove the other; commit per collision.

P1.5-WP2-T02.66 — Verify clean meta-repo + HelixAgent state

```

git submodule status --recursive | tee /tmp/p1-5-post-wp2.txt
diff <(awk '{print $2}' /tmp/p1-5-initial-shas.txt | sort) \
    <(awk '{print $2}' /tmp/p1-5-post-wp2.txt | sort)

```

Document the diff in 07_phase_1_5_evidence.md § WP2.

Commit: docs(P1.5-WP2-T02.66): WP2 evidence — submodule restructure complete.

P1.5-WP2-T02.67 — HelixAgent clean-up commit + push (deepest-first prep)

HelixAgent has the most internal changes. Push it FIRST (before its parent meta-repo) to avoid the parent pointing at SHAs only present locally.

```

cd HelixAgent
git status # must be clean
for r in $(git remote); do git push "$r" main; done
git remote | while read r; do
    printf "%-12s " "$r"; git ls-remote --heads "$r" main
done
cd ..

```

(Meta-repo push is part of WP12.)

Commit: chore(P1.5-WP2-T02.67): push HelixAgent post-restructure to all remotes.

Work Package 3 — Submodule deduplication (5 tasks)

Goal: Per the decision list (T01.04), each duplicate-set is reduced to a single canonical path; consumers of the removed paths are updated to reference the canonical via parent-traversal (`../../. . .`) or by removing the inner submodule entirely if root-level access suffices.

P1.5-WP3-T03.01 — LLMsVerifier dedup

Keep `submodules/llms_verifier`. Remove `helix_agent/LLMsVerifier`.

Pre-flight: `rg "helix_agent/LLMsVerifier" -l helix_agent/` lists every consumer. Update each to `../.. /submodules/llms_verifier` (relative from `helix_agent/<consumer>` to root). Build-test HelixAgent after the rewrite. Then:

```
cd HelixAgent
git submodule deinit -f LLMsVerifier
git rm --cached LLMsVerifier
git config -f .gitmodules --remove-section submodule.LLMsVerifier
rm -rf .git/modules/LLMsVerifier
git add .gitmodules <consumer-files>
git commit -m "chore(P1.5-WP3-T03.01): dedup LLMsVerifier;
              reference dependencies/HelixDevelopment/ canonical"
cd ..
git add HelixAgent
git commit -m "chore(P1.5-WP3-T03.01): bump HelixAgent gitlink
              after LLMsVerifier dedup"
```

P1.5-WP3-T03.02 — containers dedup

Keep root `containers/`. Enumerate the 3 nested copies. Per nested copy: `rg`, rewrite consumers, `deinit+rm`, `commit` per repo. Final HelixAgent gitlink bump.

P1.5-WP3-T03.03 — Security dedup

Keep root `security/`. 2 nested copies. Same pattern.

P1.5-WP3-T03.04 — helix_qa dedup

Keep root `helix_qa/`. Remove `helix_agent/HelixQA`. Same pattern.

P1.5-WP3-T03.05 — mcp_servers dedup (canonical TBD)

At execution time: - Check whether root `mcp_servers/` exists. If yes, keep it; remove the two HelixAgent copies. - If no: pick the more-current of the two HelixAgent copies, promote it to root via `git submodule add` at root + remove both HelixAgent entries. Document the decision in evidence.

Work Package 4 — API key loader (6 tasks)

Goal: One canonical secret-loader contract. Bash version usable from any directory in any Helix* repo. Go version usable inside any Helix* binary that needs to bootstrap its own env. Both prefer `~/api_keys.sh`, fall back to local `.env`, warn-and-continue on neither.

P1.5-WP4-T04.01 — Define loader contract

Write `docs/superpowers/specs/2026-05-06-api-keys-loader-contract.md` documenting: - Resolution order: `~/api_keys.sh` (sourced as shell with `export VAR=value` lines) → local `.env` (parsed as `VAR=value`) → no-op + warning. - `$HELIXCODE_ROOT` discovery: walk parents from cwd looking for `.gitmodules` (meta-repo) OR a `.helixcode-root` sentinel. - Mode requirements: `~/api_keys.sh` must be 0600; loader REFUSES to source a 0644 or world-readable file (CONST-042 enforcement). - Environment cleanup: loader is idempotent; sourcing twice does not duplicate-export. - Test matrix: (a) only `~/api_keys.sh` present, (b) only `.env` present, (c) both present (`api_keys.sh` wins), (d) neither (warn no-op).

Commit: `docs(P1.5-WP4-T04.01): api_keys.sh loader contract.`

P1.5-WP4-T04.02 — Implement `scripts/load_api_keys.sh`

```
#!/usr/bin/env bash
# scripts/load_api_keys.sh — canonical secret loader (CONST-042)
# Source me; do not execute. Idempotent.
load_api_keys() {
    local root="${HELIXCODE_ROOT:-}"
    if [ -z "$root" ]; then
        local d="$PWD"
        while [ "$d" != "/" ]; do
            if [ -f "$d/.gitmodules" ] || [ -f "$d/.helixcode-root" ]; then
                root="$d"; break
            fi
            d=$(dirname "$d")
        done
    fi
    if [ -f "$HOME/api_keys.sh" ]; then
        local mode
        mode=$(stat -c "%a" "$HOME/api_keys.sh" 2>/dev/null || stat -f "%A" "$HOME/api_keys.sh")
        if [ "$mode" != "600" ] && [ "$mode" != "0600" ]; then
            printf
            "load_api_keys: REFUSING ~/api_keys.sh mode=%s (must be
            600)\n" "$mode" >&2
            return 1
        fi
    fi
}
```

```

    fi
    # shellcheck source=/dev/null
    . "$HOME/api_keys.sh"
    export HELIXCODE_KEYS_SOURCE="api_keys.sh"
    return 0
fi
if [ -n "$root" ] && [ -f "$root/.env" ]; then
    set -a; . "$root/.env"; set +a
    export HELIXCODE_KEYS_SOURCE=".env"
    return 0
fi
printf "load_api_keys: no ~/api_keys.sh and no .env in %s;
        continuing without secrets\n" "${root:-CWD}" >&2
export HELIXCODE_KEYS_SOURCE="none"
return 0
}
load_api_keys

```

Commit: feat(P1.5-WP4-T04.02): scripts/load_api_keys.sh canonical bash loader.

P1.5-WP4-T04.03 — Implement Go counterpart internal/secrets/loader.go

In helix_code/internal/secrets/loader.go. TDD-first (failing tests in loader_test.go): - Load(env Env) (Source, error) where Env is an interface for Getenv / Stat / Open (so tests inject tempdirs). - Parses export VAR=value lines from ~/api_keys.sh (refuses non-0600). - Parses VAR=value lines from .env (handles # comment and quoted values). - Sets via os.Setenv only if not already set (idempotent). - Returns Source enum: SourceAPIKeysSh / SourceDotEnv / SourceNone.

Commit: feat(P1.5-WP4-T04.03): internal/secrets/loader.go — Go counterpart of bash loader.

P1.5-WP4-T04.04 — Propagate loader to every Helix* submodule

For each Helix* repo (HelixAgent, HelixQA, HelixCode-meta, dependencies/HelixDevelopment/{LLMsVerifier, DocProcessor, LLMOrchestrator, LLMProvider, VisionEngine}, plus HelixAgent's nested {HelixMemory, HelixSpecifier, HelixLLM}): copy scripts/load_api_keys.sh + ensure each repo's Makefile sources it before tests/build.

```

# Per-repo Makefile addition:
.PHONY: load-keys
load-keys:
    @. ./scripts/load_api_keys.sh

build: load-keys
    go build ./...

```

```
test: load-keys
      go test ./...
```

One commit per repo.

P1.5-WP4-T04.05 — Test loader (both branches + no-secrets branch)

helix_code/internal/secrets/loader_test.go covers: - Happy: ~/api_keys.sh mode=0600 with 2 vars → both set, source=APIKeysSh. - Refusal: ~/api_keys.sh mode=0644 → error, no env mutation. - Fallback: no ~/api_keys.sh, .env present → vars set, source=DotEnv. - Idempotent: pre-set var is NOT overwritten. - No-op: neither file → source=None, warning logged, no error.

Commit: test(P1.5-WP4-T04.05): loader exercises all 4 branches with real tempdirs.

P1.5-WP4-T04.06 — Cross-repo loader Challenge

Build challenges/p1-5-foundation-cleanup/loader-probe/main.go: tiny program that calls secrets.Load, prints (source, vars-set-count). Run it under each branch via shell harness. Capture output verbatim into 07_phase_1_5_evidence.md § WP4.

Commit: feat(P1.5-WP4-T04.06): cross-repo loader Challenge with verbatim evidence.

Work Package 5 — .env API key dedup (4 tasks; GATED)

Goal: Remove keys from .env files that are already present in ~/api_keys.sh. Keep .env files (other config may live there). Per CONST-042, this must be reversible (the user can re-add via api_keys.sh).

P1.5-WP5-T05.01 — USER GATE: confirm ~/api_keys.sh content

The agent STOPS and prints: - The list of variable names the loader resolves from ~/api_keys.sh. - Per .env file: which of those names appear there (the keys-to-remove list).

User must respond OK before proceeding. NO removal occurs without that explicit OK. Record gate decision (timestamp + response) in evidence.

(No commit — this is an interaction step.)

P1.5-WP5-T05.02 — Per-`.env` dedup

For each of the 8 `.env` files, sed-out only the lines whose key is in the `~/api_keys.sh` set:

```
KEYS_FROM_HOME=$(grep -E '^export [A-Z_]+= ' "$HOME/api_keys.sh"
| sed -E 's/^export ([A-Z_]+)=.*\/\1/')
for f in $(find . -name .env -not -path "*/.git/*"); do
  for k in $KEYS_FROM_HOME; do
    sed -i "/^${k}=/d" "$f"
  done
done
```

Commit each affected `.env` separately.

P1.5-WP5-T05.03 — Verify removal + reachability

Per affected `.env`: - Still parses cleanly (`bash -c 'set -a; . ./env; set +a'`). - Removed keys still resolve via `~/api_keys.sh` (probe with the loader Challenge from T04.06).

Capture output in evidence.

P1.5-WP5-T05.04 — Commit per affected `.env`

Already done as part of T05.02; T05.04 is the rollup commit advancing PROGRESS + writing the WP5 evidence section.

Commit: `docs(P1.5-WP5-T05.04): WP5 evidence — .env dedup complete.`

Work Package 6 — Docs consolidation (3 tasks)

Goal: One `docs/` tree per repo. No `Documentation/` directory. Update internal links.

P1.5-WP6-T06.01 — Merge root `./Documentation/` → `./docs/`

```
rsync -a Documentation/ docs/ --remove-source-files
find Documentation -type d -empty -delete
git add docs/ Documentation/
rg -l "Documentation/" --type md | xargs sed -i 's|
  Documentation/|docs/|g'
git add -A
git commit -m "docs(P1.5-WP6-T06.01): merge ./Documentation -> ./
  docs (root)"
```

P1.5-WP6-T06.02 — Merge helix_code/ Documentation/ → helix_code/docs/

Same pattern, scoped to helix_code/. Inner-module commit.

P1.5-WP6-T06.03 — Move helix_agent/skills/ development/documentation/ → helix_agent/ docs/skills/development/

```
cd HelixAgent
mkdir -p docs/skills/development
git mv skills/development/documentation docs/skills/development/
documentation
rg -l "skills/development/documentation" --type md | xargs sed
-i 's|skills/development/documentation|docs/skills/
development/documentation|g'
git add -A
git commit -m "docs(P1.5-WP6-T06.03): move HelixAgent skills
documentation under docs/"
```

Work Package 7 — Snake_case directory normalization (~10–20 tasks)

Goal: Every directory under tracked content matches `^[a-z][a-z0-9_]*$` or is in the allowlist (.git, .github, top-level repo names like HelixAgent, HelixQA, etc.; Go-convention dirs already lowercase; intentional test fixture names).

P1.5-WP7-T07.01 — Inventory non-conforming directories

```
find . -type d \
-not -path "*/.git/*" -not -path "*/node_modules/*" \
-not -path "./HelixAgent" -not -path "./HelixQA" \
-not -path "./Challenges" -not -path "./Containers" \
-not -path "./Security" -not -path "./Assets" \
-not -path "./dependencies/*" -not -path "./mcp_servers/*" \
-not -path "./Github-Pages-Website" \
-not -path "./helix_code/cmd" -not -path "./helix_code/
internal" \
| grep -E "[A-Z]|-" \
| tee docs/improvements/p1-5-snakecase-inventory.md
```

Filter the inventory: drop allowlist items; drop Go-import-path-affecting renames inside helix_code/internal/ unless trivial-case-only.

Commit: docs(P1.5-WP7-T07.01): non-conforming directory inventory.

P1.5-WP7-T07.02 .. T07.NN — Per non-conforming dir, rename + reference update

Per finding:

```
OLD=<old-path>; NEW=<new-path>
git mv "$OLD" "$NEW"
rg -l "$OLD" --type-add 'all:*' --type all | xargs -I{} sed -i
    "s|${OLD}|${NEW}|g" {}
git add -A
git commit -m "refactor(P1.5-WP7-T07.NN): snake_case rename $
    {OLD} -> ${NEW}"
```

NN advances per finding. Total ~10–20 depending on T07.01 result.

Work Package 8 — Anti-bluff Constitution propagation (3 tasks)

Goal: Every Helix* submodule's CONSTITUTION.md / CLAUDE.md / AGENTS.md carries CONST-035 + Article XI §11.9 verbatim. A verification script fails the build if the anchor is missing anywhere.

P1.5-WP8-T08.01 — Confirm anchors at meta-repo root

rg "11\\.9" CONSTITUTION.md CLAUDE.md AGENTS.md CRUSH.md QWEN.md — all must hit. Record positive evidence.

P1.5-WP8-T08.02 — Propagate to every Helix* submodule

For each Helix* repo: if CONSTITUTION.md / CLAUDE.md / AGENTS.md lacks the anchor, append the verbatim Article XI §11.9 — Anti-Bluff Forensic Anchor block (copy from root CONSTITUTION.md). If file does not exist, create from the root template. Commit per repo.

P1.5-WP8-T08.03 — Add scripts/verify-anti-bluff-cascade.sh

```
#!/usr/bin/env bash
# scripts/verify-anti-bluff-cascade.sh
# Fails if any Helix* repo's governance files lack the anti-bluff
# anchor.
set -e
ANCHOR="Article XI §11.9"
fails=0
for repo in . HelixAgent helix_qa HelixCode \
```



```

        dependencies/HelixDevelopment/
        {LLMsVerifier,DocProcessor,LLMOrchestrator,LLMProvider,VisionEngine}
    \
        helix_agent/{HelixMemory,HelixSpecifier,HelixLLM}; do
for f in CONSTITUTION.md CLAUDE.md AGENTS.md; do
    p="$repo/$f"
    [ -f "$p" ] || continue
    if ! grep -q "$ANCHOR" "$p"; then
        echo "MISSING ANCHOR: $p"
        fails=$((fails+1))
    fi
done
done
[ "$fails" = 0 ] || exit 1
echo "all governance files carry anti-bluff anchor"

```

Wire into root Makefile target `verify-anti-bluff-cascade`. Commit.

Work Package 9 — Reference updates (catch-up sweep) (2 tasks)

P1.5-WP9-T09.01 — Comprehensive grep sweep for old paths

```

for old in "Example_Projects" "Example_Resources" "helix_agent/
    cli_agents" \
    "helix_agent/LLMsVerifier" "helix_agent/HelixQA"
    "Documentation/"; do
echo "=== $old ==="
rg "$old" --type-add 'all:*' --type all -n | grep -v ".git/"
|| echo "(none)"
done | tee docs/improvements/p1-5-stale-refs.md

```

For each remaining hit: update to canonical path, commit per affected file (or batch by directory if small).

P1.5-WP9-T09.02 — Internal docs link sweep

`mkdocs.yml` (if present), Sphinx confs, README links. Update + commit.

Work Package 10 — Rebuild + validation (8 tasks)

P1.5-WP10-T10.01 — Meta-repo build + tests

```
make ci-validate-all 2>&1 | tee /tmp/p1-5-meta-ci.txt
```

P1.5-WP10-T10.02 — Inner Go module verify-compile + test

```
cd HelixCode && make verify-compile && make test 2>&1 | tee /tmp/  
p1-5-inner-test.txt  
cd HelixCode && go test -tags=integration ./... 2>&1 | tee -a /  
tmp/p1-5-inner-test.txt
```

P1.5-WP10-T10.03 .. T10.07 — Per-Helix* submodule build + test

One task per repo (HelixAgent, HelixQA, dependencies/HelixDevelopment/{LLMsVerifier,DocProcessor,LLMOrchestrator,LLMProvider,VisionEngine}). Each runs the repo's own Makefile/scripts. Failure here is a Phase 1.5 blocker.

P1.5-WP10-T10.08 — Fix pre-existing internal/tools/git mock drift (out-of-scope since F09)

Read the failing build output. The mock LLM provider in internal/tools/git/... is missing CountTokens (added to the real provider interface during F12 multi-provider work). Update the mock to match the current interface; add a unit test that asserts the mock satisfies the real interface via a var _ provider.LLMProvider = (*mockProvider)(nil) compile-time check.

Commit: fix(P1.5-WP10-T10.08): internal/tools/git mock drift — implement CountTokens to match current LLMProvider interface.

Work Package 11 — Phase 1.5 Challenge harness (4 tasks)

Goal: A real Go program that exercises every Phase 1.5 invariant with positive runtime evidence. Must fail loudly on regression.

P1.5-WP11-T11.01 — Build harness tests/ integration/cmd/p1-5-cleanup-challenge/ main.go

Five phases, each producing positive evidence (not absence-of-error):

- **Phase A — No duplicate submodules:** parse `git submodule status --recursive`; build map URL→[]paths; fail if any URL has >1 path. Print the URL→path map as evidence (positive).
- **Phase B — Loader works for both branches:** synthesise tempdir with `~/api_keys.sh mode=0600` → invoke loader → assert env vars set + source=APIKeysSh. Then synthesise tempdir with `.env` only → assert source=DotEnv. Print resolved (source, key-count) per branch.
- **Phase C — No Documentation/ anywhere:** walk tree; assert zero matches; print walked-dir count + zero-match assertion.
- **Phase D — All directories snake_case:** walk tree (with allowlist); collect non-conforming; assert zero; print conforming-count + allowlist-applied count.
- **Phase E — Anti-bluff anchor present:** call `scripts/verify-anti-bluff-cascade.sh`; assert exit 0; print per-repo OK lines.

Each phase: `t.Errorf` on failure with the exact mismatched data. No bare `t.Skip`.

P1.5-WP11-T11.02 — Challenge dir + run.sh

`challenges/p1-5-foundation-cleanup/CHALLENGE.md` with stated invariants.
`run.sh` invokes the harness; sets `HELIXCODE_ROOT`; pipes verbatim output into `docs/improvements/07_phase_1_5_evidence.md` § WP11.

P1.5-WP11-T11.03 — Run harness, capture verbatim evidence

```
cd challenges/p1-5-foundation-cleanup && ./run.sh 2>&1 \  
| tee -a ../../docs/improvements/07_phase_1_5_evidence.md
```

Must show every phase's positive evidence. Any FAIL is a Phase 1.5 blocker until fixed.

P1.5-WP11-T11.04 — Commit harness

`feat(P1.5-WP11-T11.04):` Phase 1.5 cleanup Challenge with 5-phase positive evidence.

Work Package 12 — Commit + push everything (5 tasks)

P1.5-WP12-T12.01 — Commit each submodule individually (deepest-first)

Order: 1. Each `cli_agents/<NAME>` (root-level submodules, leaves of the tree). 2. `helix_agent/HelixQA` (already removed in WP3, no-op for already-clean). 3. `helix_agent/{HelixMemory,HelixSpecifier,HelixLLM}` (any pending governance commits from WP8). 4. `HelixAgent` itself. 5. `HelixQA` root. 6. `dependencies/HelixDevelopment/*` (each). 7. Root Containers, Security, Challenges, MCP-Servers, Assets, Github-Pages-Website if any pending changes. 8. Meta-repo (last, in T12.04).

Per-submodule: `git status` clean check, then commit any pending changes.

P1.5-WP12-T12.02 — Push each submodule to ALL its configured remotes

Per submodule (in deepest-first order):

```
cd <submodule-path>
for r in $(git remote); do git push "$r" main; done
git remote | while read r; do
    printf "%-12s " "$r"; git ls-remote --heads "$r" main
done
cd -
```

Verify all remotes converge on the same SHA. Non-force only (CONST-043).

P1.5-WP12-T12.03 — Bump gitlinks at each parent

After each submodule pushes, its parent's gitlink may be ahead of what the parent's remotes have. Per parent: `git add <submodule-path>; git commit -m "chore(P1.5-WP12-T12.03): bump <submodule> gitlink"`.

P1.5-WP12-T12.04 — Push meta-repo to its 4 remotes

```
for r in github gitlab origin upstream; do git push "$r" main;
done
for r in github gitlab origin upstream; do
    printf "%-10s " "$r"; git ls-remote --heads "$r" main
done
```

All four must show the same SHA.

P1.5-WP12-T12.05 — Verify every remote has the new SHA

Final pass over every repo + every remote. Any divergence is a Phase 1.5 blocker.

Final commit (meta-repo): `chore(P1.5-WP12-T12.05): Phase 1.5 — Foundation Cleanup COMPLETE; all remotes synced.`

Update PROGRESS.md: Phase 1.5 closed; active phase advances to Phase 2.

Final acceptance checklist (must ALL be ticked before claiming Phase 1.5 done)



All 12 WPs commits landed. (WP1 through WP12 — see PROGRESS.md P1.5 work-package list for SHAs)



Phase 1.5 Challenge harness exit 0 with all 5 phases printing positive evidence. (`/tmp/p1_5_challenge EXIT=0` captured in evidence §P1.5-WP12)



`scripts/verify_anti_bluff_cascade.sh` exit 0. (“OK: anti-bluff anchor present in all 39 files across 13 repos”)



`git submodule status --recursive` shows each URL at exactly one path. (Phase A of harness)



No Documentation/ directory anywhere. (Phase C of harness)



Every directory `snake_case` (or in allowlist). (Phase D of harness — 259 conformant, 88 allowlisted, 0 violations)



Every `.env` has no overlap with `~/api_keys.sh` keys (post-WP5 USER GATE). (See §P1.5-WP5 evidence)



`cd HelixCode && make verify-compile && make test` passes. (Inner unit tests `go test -count=1 -short ./internal/... ./cmd/...` all green; see §P1.5-WP12)



`internal/tools/git mock drift fixed` (T10.08). (Commit 45be827)



All meta-repo remotes (github/gitlab/origin/upstream) on same SHA. (See §P1.5-WP12 push verification table)



Each Helix* submodule pushed to ALL its configured remotes. (See §P1.5-WP12 per-submodule push status table)



PROGRESS.md advanced to Phase 2.



Anti-bluff smoke at end of every task printed `clean`.

Phase 1.5 is the bridge. Build it once; build it clean.